INFRASTRUCTURE / PORTFOLIO CASE STUDY

NGINX Reverse Proxy Setup

A single NGINX front door for every internal service — clean subdomain routing, automatic HTTP → HTTPS with certbot-managed TLS, per-client rate limiting, and security hardening. One hostname and port pair per service is replaced by one memorable domain name on port 443.

NGINX	**TLS / HTTPS**	**certbot (ACME)**	**Rate limiting**
Subdomain routing	**Bash**	**systemd**	**Linux**

Environment
Northstar Animation Studios — Linux pipeline

Proxy host
Single NGINX edge node

Services fronted
AWX, Grafana, render farm, wiki, registry

Summary. Internal services tend to sprawl across raw IP:port pairs — `10.20.0.11:8080` for one tool, `10.20.0.30:3000` for another — all unencrypted and impossible to remember. This project consolidates them behind one NGINX reverse proxy. Each service gets a clean subdomain (for example `awx.northstar.studio`), every request is served over HTTPS with certificates issued and renewed automatically by certbot, abusive clients are throttled with per-IP rate limiting, and a shared set of security headers is applied across the board. The proxy also terminates TLS once and caches responses, cutting repeated round-trips to the backends. The result: services are reachable by name, encrypted by default, and protected by a single, auditable configuration layer.

1 — Why a Reverse Proxy

A reverse proxy sits in front of one or more backend services and speaks to clients on their behalf. Instead of exposing each application on its own host and port, every request enters through a single edge node, which decides where it should go and how it should be treated. For a studio running a dozen internal web tools, this collapses a messy set of `IP:port` endpoints into one tidy, encrypted, policy-controlled surface.

What the proxy provides

- **Name-based access.** `grafana.northstar.studio` instead of `10.20.0.30:3000` — the IP and port are hidden behind a domain.
- **One TLS termination point.** HTTPS is handled at the edge; backends can stay plain HTTP on the trusted internal network, and certificates live in one place.
- **Rate limiting & protection.** Per-client request limits absorb bursts and misbehaving scripts before they reach a backend.
- **Reduced traffic.** Connection reuse to upstreams, gzip compression, and response caching cut repeated round-trips and bytes on the wire.
- **One config to audit.** Routing, headers, and security policy live in a single version-controlled directory rather than scattered across apps.

Request flow

1	**Client request**	Browser asks for https://awx.northstar.studio (port 443).
2	**TLS termination**	NGINX presents the certbot-issued certificate and decrypts the request.
3	**Rate-limit check**	The request is counted against the per-IP zone; bursts beyond the limit are delayed or rejected.
4	**Route by host**	NGINX matches the server_name and selects the matching upstream.
5	**Proxy to backend**	The request is forwarded over HTTP to the internal service with forwarded headers set.
6	**Response**	The backend reply is (optionally) compressed/cached and returned to the client over HTTPS.

Service map

Each internal tool is reached through its own subdomain on the proxy. The backends themselves never need to be touched by clients directly.

Subdomain	Backend (upstream)	Service
`awx.northstar.studio`	`10.20.0.11:8080`	Ansible AWX control node
`grafana.northstar.studio`	`10.20.0.30:3000`	Metrics dashboards
`farm.northstar.studio`	`10.20.0.40:8082`	Render farm monitor
`wiki.northstar.studio`	`10.20.0.50:80`	Internal documentation wiki
`registry.northstar.studio`	`10.20.0.60:5000`	Container image registry

2 — Core NGINX Setup

NGINX is installed on the edge node and configured with one file per service under `conf.d/`. This keeps each subdomain self-contained and easy to add, remove, or review independently.

2.1 — Install and open the firewall

```
dnf install -y nginx
systemctl enable --now nginx

# allow HTTP (for ACME challenge + redirect) and HTTPS
firewall-cmd --add-service={http,https} --permanent
firewall-cmd --reload
```

2.2 — Layout

Shared settings (proxy headers, rate-limit zones, TLS parameters) live in include files; each service block then pulls them in with a single line.

```
/etc/nginx/
  nginx.conf # global: limit_req zones, gzip, logging
  conf.d/
    awx.conf # one server block per subdomain
    grafana.conf
    farm.conf
  snippets/
    proxy-headers.conf # shared proxy_set_header block
    tls-params.conf # shared SSL protocols / ciphers
    security-headers.conf # HSTS, X-Frame-Options, etc.
```

2.3 — Shared proxy headers

Backends need to know the real client address and the original scheme. These headers are defined once and included by every service block.

```
# /etc/nginx/snippets/proxy-headers.conf
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_http_version 1.1;
proxy_set_header Connection ""; # enable keepalive to upstream
```

2.4 — A service block

A single upstream and server block defines one subdomain. An `upstream` with keepalive lets NGINX reuse connections to the backend instead of opening a new one per request — one of the ways the proxy reduces traffic.

```
# /etc/nginx/conf.d/awx.conf
upstream awx_backend {
    server 10.20.0.11:8080;
    keepalive 16;
}

server {
    listen 443 ssl;
    server_name awx.northstar.studio;

    include snippets/tls-params.conf;
    include snippets/security-headers.conf;

    location / {
        limit_req zone=perip burst=20 nodelay;
        include snippets/proxy-headers.conf;
        proxy_pass http://awx_backend;
    }
}
```

3 — HTTPS & Automated TLS

Every subdomain is served over HTTPS. Certificates are obtained and renewed automatically by certbot, and all plain-HTTP traffic is permanently redirected to HTTPS so nothing is ever served in the clear.

3.1 — Issue certificates with certbot

For services on a real, DNS-resolvable domain, certbot's NGINX plugin requests a certificate and wires it into the relevant server block automatically.

```
dnf install -y certbot python3-certbot-nginx

# issue + auto-configure certs for each subdomain
certbot --nginx \
  -d awx.northstar.studio \
  -d grafana.northstar.studio \
  -d farm.northstar.studio
```

A wildcard certificate is cleaner when many subdomains share one domain. Wildcards require the DNS-01 challenge, which certbot automates through a DNS provider plugin:

```
# one wildcard cert covering every *.northstar.studio service
certbot certonly --dns-cloudflare \
  --dns-cloudflare-credentials /etc/letsencrypt/dns.ini \
  -d "northstar.studio" -d "*.northstar.studio"
```

3.2 — Automatic renewal

Let’s Encrypt certificates last 90 days. certbot installs a systemd timer that renews them well before expiry and reloads NGINX on success — no manual intervention, no expired certs in production.

```
# certbot ships a renewal timer; just confirm it is active
systemctl enable --now certbot-renew.timer
systemctl list-timers certbot-renew.timer

# dry-run to prove renewal works end to end
certbot renew --dry-run

# reload NGINX automatically after a successful renew
# /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
#!/usr/bin/env bash
systemctl reload nginx
```

3.3 — Redirect HTTP to HTTPS

A small port-80 server block catches any plain-HTTP request and issues a permanent redirect, while still allowing the ACME challenge path through for renewals.

```
server {
    listen 80;
    server_name *.northstar.studio;

    # let certbot's HTTP-01 challenge through
    location /.well-known/acme-challenge/ { root /var/www/acme; }

    # everything else: force HTTPS
    location / { return 301 https://$host$request_uri; }
}
```

3.4 — Internal-only services (local CA)

Some tools are never exposed to the public internet and may sit on a domain that certbot cannot validate. For those, an internal certificate authority issues certificates that the studio's machines trust — the same HTTPS termination, served from a locally-signed cert instead of a public one.

```
# create a long-lived internal CA (once)
openssl req -x509 -newkey rsa:4096 -nodes -days 3650 \
  -keyout internal-ca.key -out internal-ca.crt \
  -subj "/CN=Northstar Internal CA"

# issue a cert for an internal subdomain, signed by that CA
openssl req -newkey rsa:2048 -nodes \
  -keyout repo.key -out repo.csr \
  -subj "/CN=registry.northstar.studio"
openssl x509 -req -in repo.csr -CA internal-ca.crt \
  -CAkey internal-ca.key -CAcreateserial \
  -out repo.crt -days 825
```

The internal CA certificate is distributed to client machines (conveniently, via the Ansible fleet from the previous project) so internal HTTPS shows as trusted everywhere.

4 — Rate Limiting & Hardening

4.1 — Per-client rate limiting

A rate-limit zone tracks requests per client IP. The zone is declared once at the global level and referenced inside the service blocks. `burst` absorbs short spikes; `nodelay` serves the burst immediately rather than queuing it.

```
# /etc/nginx/nginx.conf (http context)
limit_req_zone $binary_remote_addr zone=perip:10m rate=10r/s;
limit_req_status 429;

# referenced inside a location block:
# limit_req zone=perip burst=20 nodelay;
```

At 10 requests/second with a burst of 20, a normal user never notices, while a runaway script or scraper is capped and receives `429 Too Many Requests` instead of overwhelming a backend.

4.2 — Security headers

A shared snippet applies a baseline of response headers to every service — enforcing HTTPS, blocking clickjacking, and disabling content-type sniffing.

```
# /etc/nginx/snippets/security-headers.conf
add_header Strict-Transport-Security "max-age=63072000" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
server_tokens off; # hide the NGINX version banner
```

4.3 — TLS parameters

Modern protocols and ciphers only, with session caching to keep repeat handshakes cheap.

```
# /etc/nginx/snippets/tls-params.conf
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
ssl_certificate /etc/letsencrypt/live/northstar.studio/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/northstar.studio/privkey.pem;
```

4.4 — Compression and caching

gzip shrinks text responses on the wire, and a small proxy cache for static assets means repeated requests are answered by NGINX without touching the backend at all — directly addressing the traffic-reduction goal.

```
gzip on;
gzip_types text/plain text/css application/json application/javascript;
gzip_min_length 1024;

# cache static upstream responses briefly
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=assets:10m max_size=1g;
```

4.5 — Validate before reload

Every change is tested before it goes live, so a typo never takes the proxy down.

```
nginx -t # syntax + config test
systemctl reload nginx # graceful reload, no dropped connections
```

5 — Outcomes

Before	After
Services reached by raw IP:port; nothing memorable, easy to misroute.	Each service has a clean subdomain; the IP and port are hidden behind the proxy.
Internal tools served over plain HTTP — credentials and data in the clear.	Everything served over HTTPS; HTTP is permanently redirected.
Certificates (where they existed) renewed by hand and sometimes expired.	certbot issues and auto-renews via systemd timer; zero manual steps.
No protection against bursty or abusive clients hitting a backend.	Per-IP rate limiting caps bursts and returns 429 before backends are hit.
Security headers and TLS settings configured ad hoc, per app.	One shared config applies headers, ciphers, and caching uniformly.

Technical stack

Layer	Technology
Reverse proxy	NGINX (one server block per subdomain)
TLS / certificates	certbot + Let's Encrypt (ACME); internal OpenSSL CA for local-only
Challenge types	HTTP-01 (per-host) and DNS-01 (wildcard)
Renewal	systemd timer (certbot-renew) with NGINX deploy hook
Rate limiting	limit_req_zone (per-IP), 429 on exceed
Hardening	HSTS, X-Frame-Options, nosniff, TLS 1.2/1.3 only
Performance	upstream keepalive, gzip, proxy_cache
Automation	Bash, validated reloads (nginx -t)
OS / service mgmt	Linux (Rocky), systemd

Names, hostnames, domains, and addresses in this document are anonymized for portfolio use. The architecture, tooling, and workflow reflect a real production deployment.